

template<typename D>

class LIST {

node<D>* H;

node<D>* T;

long int elem;

// "Iterator class is needed to wrap raw pointers and provide controlled access with operator overloading so the container behaves like STL instead of exposing internal structure."

public:

// "We use iterator instead of returning raw pointer because pointer exposes internal node structure and cannot be controlled, while iterator provides abstraction and operator-based access."

class iterator {

node<D>* ptr;

public:

iterator(node<D>* p = nullptr) : ptr(p) {} *// minimum required*

D& operator*(){ *// minimum required*

if(!ptr) throw std::runtime_error("Null iterator");

return ptr->K;

}

const D& operator*() const {

if(!ptr) throw std::runtime_error("Null iterator");

return ptr->K;

}

"bad design" -> Postfix is a "derived / extra behavior"

iterator& operator++() noexcept {

if(ptr) ptr = ptr->P;

return *this; *// minimum required*

}

iterator operator++(int) noexcept {

iterator temp = *this;

++(*this);

return temp;

}

iterator& operator++() noexcept {

if(ptr) ++(*this);

return *this;

}

iterator operator++(int) noexcept {

iterator temp = *this;

if(ptr) ptr = ptr->P;

return temp;

}

// minimum required

bool operator!=(const iterator& other) const noexcept { return ptr != other.ptr; }

bool operator==(const iterator& other) const noexcept { return ptr == other.ptr; }

D* operator->(){ return &(ptr->K); }

const D* operator->() const { return &(ptr->K); }

// string s = "hello"; string p = &s;*

// cout << p->size(); we cannot overload (.) in C++

};

// thats why Overload (->)

```
operator->() Notes
struct Person { std::string name; };

LIST<Person> people;
auto it = people.begin();

Person* operator->() { return &(ptr->K); }

When you write: it->name;
the compiler automatically rewrites it as:
it.operator->()->name;
```

There are two -> operators:

1. **it.operator->()** → Calls your overloaded operator->().
2. **->name** → Uses the built-in pointer -> operator.

Your **operator->()** returns: **Person* p = &(ptr->K);**

Then: **p->name;** is equivalent to: **(*p).name;**

The built-in pointer
-> automatically dereferences the pointer, so you do not write * yourself.

Rule: operator->() must return a pointer (or another object that also overloads operator->()).

```
//... .
```

```
iterator begin() noexcept { return iterator(H); } // minimum required
iterator end() noexcept { return iterator(nullptr); } // minimum required
```

```
iterator begin() const noexcept { return iterator(H); }
iterator end() const noexcept { return iterator(nullptr); }
```

```
};
```

```
int main(){
    LIST<int> l;

    l.insert(10);
    l.insert(20);
    l.insert(30);

    for(auto it = l.begin(); it != l.end(); ++it){
        cout << *it << " ";
    }

    for(auto x : l){
        cout << x << " ";
    }

    // for(auto x : l) 👉 Compiler converts it into
    // auto && __range = l;
    // for(auto __it = __range.begin(); __it != __range.end(); ++__it){
    //     auto x = *__it;
    // }

}
```

```

template<typename D>
struct BSTnode{ D K; BSTnode<D>** LINK; };

template<typename D>
class BSTiterator {
    BSTnode<D>* ptr, *ROOT, *last;
    STACK<BSTnode<D>*> S;
    QUEUE<BSTnode<D>*> Q;
public:
    MODE M;
    BSTiterator(MODE mode, BSTnode<D>* p = nullptr);
    BSTnode<D>* increment();
    BSTnode<D>* incrementBFS();
    BSTnode<D>* incrementDFS();
    BSTnode<D>* incrementIN();
    BSTnode<D>* incrementPRE();
    BSTnode<D>* incrementPOST();
    D& operator*();
    const D& operator*() const;
    BSTiterator& operator++() noexcept;
    BSTiterator operator++(int) noexcept;
    bool operator!=(const BSTiterator& other) const noexcept;
    bool operator==(const BSTiterator& other) const noexcept;
    D* operator->();
    const D* operator->() const;
};

```

```

template<typename D>
BSTiterator<D>::BSTiterator(MODE mode, BSTnode<D>* p) : ptr(nullptr), last(nullptr), ROOT(p), M(mode) {
    if(!ROOT) return;

    switch(M){
        case MODE::BFS:
        case MODE::BFS_RL:
            Q.enqueue(ROOT); break;

        case MODE::DFS:
        case MODE::DFS_LR:

        case MODE::PRE:
            S.push(ROOT); break;

        case MODE::IN:
        case MODE::POST:
        {
            BSTnode<D>* temp = ROOT;
            while(temp){
                S.push(temp);
                temp = temp->LINK[L];
            } break;
        }

        default: break;
    }
}

```

```

template <typename D>
BSTNode<D>* BSTIterator<D>::incrementBFS(){
    if(Q.empty()) return nullptr;
    BSTNode<D>* temp = Q.dequeue();

    if(M == MODE::BFS){
        if(temp->LINK[L]) Q.enqueue(temp->LINK[L]);
        if(temp->LINK[R]) Q.enqueue(temp->LINK[R]);
    }else{
        if(temp->LINK[R]) Q.enqueue(temp->LINK[R]);
        if(temp->LINK[L]) Q.enqueue(temp->LINK[L]);
    }
    return temp;
}

```

```

template<typename D>
BSTNode<D>* BSTIterator<D>::incrementDFS(){
    if(S.empty()) return nullptr;
    BSTNode<D>* temp = S.pop();

    if(M == MODE::DFS){
        if(temp->LINK[L]) S.push(temp->LINK[L]);
        if(temp->LINK[R]) S.push(temp->LINK[R]);
    }else{
        if(temp->LINK[R]) S.push(temp->LINK[R]);
        if(temp->LINK[L]) S.push(temp->LINK[L]);
    }
    return temp;
}

```

```

template<typename D>
BSTNode<D>* BSTIterator<D>::incrementIN(){
    if(S.empty()) return nullptr;
    BSTNode<D>* node = S.pop();

    BSTNode<D>* tmp = node->LINK[R];
    while(tmp){
        S.push(tmp);
        tmp = tmp->LINK[L];
    }
    return node;
}

```

```

template<typename D>
BSTNode<D>* BSTIterator<D>::incrementPRE(){
    if(S.empty()) return nullptr;
    BSTNode<D>* node = S.pop();

    if(node->LINK[R]) S.push(node->LINK[R]);
    if(node->LINK[L]) S.push(node->LINK[L]);
    return node;
}

```

```

template<typename D>
BSTNode<D>* BSTIterator<D>::incrementPOST(){
    while(!S.empty()){
        BSTNode<D>* peek = S.tos();
        BSTNode<D>* right = peek->LINK[R];

        if(right && last != right){
            BSTNode<D>* tmp = right;
            while(tmp){
                S.push(tmp);
                tmp = tmp->LINK[L];
            }
        }else{
            S.pop();
            last = peek;
            return peek;
        }
    }
    return nullptr;
}

```

```

template<typename D>
BSTNode<D>* BSTIterator<D>::increment(){
    switch(M){
        case MODE::BFS:
        case MODE::BFS_RL:
            return incrementBFS();

        case MODE::DFS:
        case MODE::DFS_LR:
            return incrementDFS();

        case MODE::IN: return incrementIN();
        case MODE::PRE: return incrementPRE();
        case MODE::POST: return incrementPOST();

        default: return nullptr;
    }
}

```

```
template<typename D>
D& BSTiterator<D>::operator*(){
    if(!ptr) throw std::runtime_error("Null iterator");
    return ptr->K;
}
```

```
template<typename D>
const D& BSTiterator<D>::operator*() const{
    if(!ptr) throw std::runtime_error("Null iterator");
    return ptr->K;
}
```

```
template<typename D>
D* BSTiterator<D>::operator->(){
    if(!ptr) throw std::runtime_error("Null iterator");
    return &(ptr->K);
}
```

```
template<typename D>
const D* BSTiterator<D>::operator->() const{
    if(!ptr) throw std::runtime_error("Null iterator");
    return &(ptr->K);
}
```

```
template<typename D>
BSTiterator<D>& BSTiterator<D>::operator++() noexcept{
    ptr = increment();
    return *this;
}
```

```
template<typename D>
BSTiterator<D> BSTiterator<D>::operator++(int) noexcept{
    BSTiterator<D> temp = *this;
    ptr = increment();
    return temp;
}
```

```
template<typename D>
bool BSTiterator<D>::operator!=(const BSTiterator<D>& other) const noexcept{ return ptr != other.ptr; }
```

```
template<typename D>
bool BSTiterator<D>::operator==(const BSTiterator<D>& other) const noexcept{ return ptr == other.ptr; }
```

```
template<typename D>
class Cacheliterator {
    D* cache;
public:
    Cacheliterator() { cache = new D[100]; }
    ~Cacheliterator() { delete[] cache; }
};
```

Destructor Needed

A custom destructor is needed when the iterator directly owns resources such as dynamically allocated memory (new), file handles, sockets, or other system resources. The destructor must explicitly release these resources to prevent memory leaks or resource leaks.

```
template<typename D>
BSTiterator<D>::~BSTiterator()
{
    Q.~QUEUE<BSTnode<D>*>();
    S.~STACK<BSTnode<D>*>();
}
```

Destructor Not Needed

A custom destructor is not needed when the iterator does not own any resources. The compiler automatically destroys all member objects (such as STACK, QUEUE, and LIST) and calls their destructors in reverse order when the iterator object is destroyed.

```
template<typename D>
class STACK;
```

```
template<typename D>
union LIFO
{
```

```
    struct Continuous { friend class STACK<D>;
```

```
    protected:
```

```
        ::ARRAY<D> S;
```

```
    public:
```

```
        Continuous(size_t size=0) :S(size) {}
```

```
        size_t size();
```

```
        bool empty();
```

```
        bool push(D val);
```

```
        D tos();
```

```
        D pop();
```

```
        void view(bool v) const;
```

```
        void clear();
```

```
        Continuous(const Continuous&) = default;
```

```
        Continuous& operator=(const Continuous&) = default;
```

```
        Continuous(Continuous&&) noexcept = default;
```

```
        Continuous& operator=(Continuous&&) noexcept = default;
```

```
    } array;
```

```
    struct Linked { friend class STACK<D>;
```

```
    protected:
```

```
        ::LIST<D> S;
```

```
    public:
```

```
        Linked()=default;
```

```
        size_t size();
```

```
        bool empty();
```

```
        bool push(D val);
```

```
        D tos();
```

```
        D pop();
```

```
        void view(bool v) const;
```

```
        void clear();
```

```
        Linked(const Linked&) = default;
```

```
        Linked& operator=(const Linked&) = default;
```

```
        Linked(Linked&&) noexcept = default;
```

```
        Linked& operator=(Linked&&) noexcept = default;
```

```
    } list;
```

```
LIFO() {}
```

```
~LIFO() {}
```

```
LIFO(const LIFO&)=delete;
```

```
LIFO& operator=(const LIFO&)=delete;
```

```
LIFO(LIFO&&)=delete;
```

```
LIFO& operator=(LIFO&&)=delete;
```

```
};
```

✓ When you write = default for a copy or move constructor (or assignment operator), you are telling the C++ compiler to automatically create that function. The compiler does not simply copy the raw memory; instead, it copies or moves each member one by one. If a member, such as ARRAY or LIST, has its own custom copy or move constructor, the compiler automatically calls that custom function. So, = default means "generate the function for me," while each member still uses its own copy or move logic if it has one.

- A house (LIFO) contains two rooms (Continuous and Linked).
- You can say, "The whole house cannot be moved."
- That doesn't mean "The furniture inside each room cannot be moved."

✓ LIFO deletes its copy and move constructors and assignment operators to prevent the entire union object from being copied or moved. A union can contain only one active member (array or list) at a time, so disabling these operations avoids accidental misuse. This does not affect the Continuous or Linked classes inside the union, because they are separate types with their own copy and move functions. They can still use their own = default constructors and assignment operators or their custom implementations independently of LIFO.